OG Companion

Technical Specification

Version 2.1.0 | Rob Johnston | Prepared for Alex Richards | 09/10/2026

Purpose

I built OG Companion to give learners a practical place to start when they are new to coding. Someone can describe a program, explore the libraries behind its features and work with real editable examples. They can inspect an operation, adapt it and develop their own implementation rather than having to know every package name before they begin.

Version 2.1 adds clear-search and clear-code controls, repeatable expansion, updates using the current code and undo. The source browser offers on-demand code retrieval from package descriptions, README files and documentation, a complete DevDocs edition and API browser, public repository discovery, source-URL loading and paged results. The bundled teaching examples remain useful offline, but no longer define the collection of code someone can access.

Item	Specification
Product	OG Companion 2.1.0, OG Web.site branding
Platform	Portable Windows 64-bit desktop application
Interface	Tkinter/ttk, editable code workbench and source selector
Runtime	Bundled Python 3.14.0 and Tcl/Tk; PyInstaller 6.17.0
Discovery	Seven package registry connectors, DevDocs and public GitHub repositories
Documentation	244 distinct collection titles across 837 editions; manifest included, content fetched on demand
Examples	Retrieved on demand; every distinct block from the loaded pages is selectable
Local foundations	30 teaching entries and one CSV-to-JSON program
Custom generation	Optional local Ollama model; service and weights supplied separately

The checked PyPI index contained 909,023 published package names on 08/10/2026. This is a search index, not installed libraries or bundled programs. The full index is downloaded per recipient and is not in the ZIP. The content-inventory.json asset records exact bundled counts, collection titles and checked source-block counts.

There is no fixed total of available snippets. Availability depends on the chosen public sources: some pages contain no code and providers can impose search, traffic or access limits. The application does not claim to contain every library or every possible program.

Discovery and documentation workflow

Choose a language and describe the required inputs, operations and outputs. The optional keyword field narrows discovery without replacing the full program description. Find libraries searches the selected source; short library names or API terms generally give more precise results than a whole sentence.

Source mode	Behaviour
Packages	Python searches the full PyPI name index. JavaScript/TypeScript use npm; C#/F# use NuGet; Java/Kotlin/Scala use Maven Central; Rust uses crates.io; PHP uses Packagist; Ruby uses RubyGems. Other languages use public repository discovery.
Documentation	Select an edition from the full published manifest, then search its API index or choose All API topics. Previous and Next results reach further index entries.
GitHub	Search public repositories using the request keywords and language qualifier. Repository README code can be loaded into the workbench.
Source URL	Load another public HTTPS documentation page, raw README or GitHub repository independently of the search connectors.

Clear search and retain your work

Clear search empties the description, keywords, library results and example selection while retaining code. Late results from a cleared search are discarded. Clear code separately empties the editor after confirmation for unsaved changes. Selecting a library changes its details; Get code examples explicitly loads its examples.

Examples beside the library

Get code examples reads the selected topic or library sources. Package retrieval checks registry descriptions and README metadata where supported, then up to four available documentation or repository addresses. Every distinct code block found in those loaded documents is retained. A source without readable code blocks produces an explicit empty result.

Markdown fences, reStructuredText code directives and HTML preformatted blocks are supported. Declared language hints are retained. Commands, configuration and Python interactive transcripts have separate labels; a transcript is not presented as an executable Python script. Sites requiring authentication or client-side rendering may need a different public usage page.

The example selector opens a block as editable text. Copy code includes edits. Save code uses the displayed code language. Save all examples exports the loaded collection as JSON with code, language, source URL and attribution/licence details. External copyright and licence conditions still apply.

Architecture and data contracts

Component	Responsibility
og_companion.py	OGCompanion window, source/edition/topic controls, paging, workbench, copy/save/export, asynchronous result handling and installation check
og_companion_search.py	Offline ranking, standard-library discovery, registry adapters, PyPI index cache, model discovery and code-generation validation
og_companion_sources.py	Public URL checks, text fetching, document caching, code extraction, DevDocs topic discovery, GitHub search and README retrieval
og_companion_library.json	Editable offline teaching examples, search tags and CSV-to-JSON foundation
og_companion_assets	Genuine OG logo/icon and documentation-index.json manifest
OG Runtime	Bundled interpreter, Tcl/Tk, native runtime libraries and licences

Results and examples

A library result includes name, language, languages, source, description, version, docs, registry_url, install, snippet and tags. Documentation results add their edition metadata, topic path and attribution. An extracted example includes title, code, language, url, attribution and licence; retrieval timestamps are included for extracted source blocks.

DevDocs supplies a manifest of documentation editions and an index of entries with name, path and type. The application downloads an index when an edition is searched and fetches the selected topic page when its examples are requested. It does not download every documentation database at startup.

UI coordination

Network and model operations run in background workers. The Tk event loop reads results every 100 milliseconds. Package documentation retrieval uses at most three parallel readers. Selection tokens prevent late Python metadata from replacing a newer selection. Worker request identifiers discard cleared search results and progress. Search settings and code-generation inputs are captured per request. Pending callbacks are cancelled when the window closes.

Unsaved editor content is protected before replacement or closing. If newer edits appear while generation is running, replacement needs confirmation. Selecting another library changes its details while retaining the current code. The installation resolves resources beside the executable or source file, rather than through development-machine paths.

Network interfaces and caching

Interface	Use and paging
PyPI	JSON simple index for all package names; package JSON for metadata and description. Name matches are sliced into pages of 50.
npm	Registry search with size=50 and from offset; package metadata supplies README/repository data.
NuGet	Discovered SearchQueryService with take=50, skip offset and prerelease=false.
Maven Central	Sonatype Solr search with rows=50, zero-based page index in start, and wt=json.
crates.io	Crate search with per_page=50 and page; crate metadata for repository lookup.
Packagist / RubyGems	Packagist uses per_page=50 and page; RubyGems uses its provider-defined page size and page parameter.
DevDocs	devdocs.io/docs.json; documents.devdocs.io/{edition}/index.json and selected topic HTML; 100 API entries displayed per page.
GitHub	Public repository search with per_page=50 and page; repository README metadata and its public raw download URL.

A page size is a display/request size, not a total code limit. Provider limits still apply, including maximum searchable windows and unauthenticated GitHub rate limits. An unavailable service is reported rather than treated as an empty complete library.

Local cache

PyPI names and their fetched_at timestamp are gzip-compressed under %USERPROFILE%\.og-companion. A process lock serialises index refreshes and unique temporary files prevent simultaneous writers sharing a temporary path. Fresh names are reused for 24 hours; a retained stale index is used if refresh fails.

Text documents are cached as JSON under the documents subfolder using SHA-256 URL filenames. Records contain text, final URL, content type and fetched_at. A fresh document is reused for 24 hours. Offline mode uses already downloaded documents; an uncached page requests online access. The bundled manifest remains available if catalogue refresh fails.

Default registry requests have a 25-second timeout and 12 MB decoded-response limit; the complete PyPI index allows 60 seconds and 160 MB. Documentation requests allow 25 seconds and 8 MB per text response. Large or unreadable pages require a more specific source page.

Starter programs and local generation

An editable foundation

The supplied CSV-to-JSON program demonstrates input, validation, display and output. csv.DictReader reads the actual headers. Rows with missing, blank or whitespace-only cells, or extra unnamed fields, are rejected. Valid records and counts are printed and accepted records are written as a JSON list. It refuses to overwrite its input and handles empty or missing files.

Build / update code selects the foundation only for an empty editor and a supported request. With existing code, it sends that code and the changed description for an update. Expand code starts another request on every click after the preceding request finishes. Undo update restores the preceding editor change. The foundation is selected only when its supported task vocabulary covers the request. Other program requests use the selected local model. Source examples can be used directly without AI, and their context must be checked before they are incorporated into a larger program.

Model interface	Specification
Discovery	GET http://127.0.0.1:11434/api/tags; four-second timeout
Generation	POST http://127.0.0.1:11434/api/generate, non-streaming
Request	model, system, prompt, options and stream; source-code response requested
Options	Temperature 0.2; up to 3,200 predicted tokens for a program or 1,800 for a focused snippet
Context	Selected library metadata, up to three loaded matching-language examples, current editor code, previous requirement and expansion round; over 24,000 characters rejected without truncation
Validation	Empty, incomplete or output-limit responses rejected; Python syntax parsed; identical updates rejected; Python comment-only updates and dropped top-level function/class names rejected
Repair	One repair for invalid syntax or rejected expansion; 300-second timeout per generation request

Returned source is extracted from plain text, Markdown fences or a legacy JSON code field. Retrieved metadata and examples are identified as untrusted reference data. The application asks for reusable functions, imports, requested interaction, useful comments and coverage of each feature; these instructions do not prove the generated behaviour.

Ollama and model weights are not redistributed in the portable folder. Recipients install their own model for custom generation. Small models can return incomplete or faulty logic, and no compiler gate is included for languages other than Python. The application never runs model output.

Distribution, privacy and source terms

Portable delivery

The folder contains the executable, launcher, Read Me, this specification, editable application modules and tests, offline catalogue, assets and OG Runtime. Extract the ZIP and keep the complete folder together. Windows recipients do not need a separate Python installation to open the executable.

Source execution needs Python 3 with Tcl/Tk and no additional pip packages. Saved programs require their own language runtimes and dependencies. Changes to Python source need an executable rebuild; JSON catalogue and asset changes are loaded from the folder by both source and packaged execution.

Public sources and privacy

Online discovery sends search terms to the chosen public source. Documentation loading sends requests to the selected public page or registry/repository endpoint. Source fetching accepts public HTTPS text pages and checks redirect destinations; local/private addresses, embedded credentials and nonstandard ports are rejected. Documents are treated as data, never instructions or executable application content.

Custom generation sends the task and reference/expansion code to the local Ollama service. There is no cloud AI account, shipped API key, central learner database or analytics. Workbench content remains in memory unless copied or saved; there is no automatic session recovery. Browser visits are separate from the online-search setting.

Attribution and limitations

DevDocs, maintained by freeCodeCamp, supplies documentation metadata and content from original documentation projects. The application retains edition attribution and example source URLs. Registry README examples retain supplied licence information; where a licence is absent, the source terms must be checked. This is not an automatic licence audit or a declaration that all retrieved code is unrestricted.

There is no package installation, automatic execution, vulnerability scan, sandbox or code-signing certificate. The application does not certify downloaded packages or generated code as safe. No macOS, Linux or ARM64 portable executable has been verified. Model performance and hardware requirements depend on the chosen model.

Verification and recipient checks

Checked inventory: 30 offline snippets across 14 languages and one complete foundation; 244 documentation collection titles across 837 editions; 21 selector choices; seven registry connectors covering 11 choices directly. No complete global online snippet count has been measured. Source-block counts describe loaded pages, not a total collection of complete programs.

The suite checks discovery, pagination, source extraction, cached offline access, language labels, private-address rejection, response parsing and Python validation. A sample page retains all 83 distinct blocks.

Check	Evidence and boundary
Source tests	72 automated tests passed, including repeat clicks, changed requirements, clearing, stale results, undo, language guards and failure preservation.
Live packages	httpx, Rich, Polars, Axios, Commander and Tokio yielded source examples outside the offline catalogue.
Example volume	Axios returned 105 distinct examples; Commander 67. The code does not impose a 30-example cap.
API documentation	Python CSV returned 16 examples; a Go topic page 204; JavaScript Date.toJSON four. These are extracted source blocks, not executed-program results.
Own-app interaction	Callbacks checked discovery, examples, save/export and paging. Live model checks passed two expansions, a changed requirement, undo and positive, negative, zero and decimal arithmetic cases.
Portable package	Installation and relocation checks verify bundled GUI resources, manifest, source-browser controls and the supplied foundation; ZIP contents are hash-checked.

All seven package-registry connectors returned different result sets for their first and second pages in live checks. One actual locally generated console program passed controlled tests for valid input, rejection of a negative age followed by valid input, and rejection of a noninteger age followed by valid input. Earlier model failures were observed; these checks establish that example only.

Code correctness boundary

The supplied CSV foundation is executed in tests against valid and invalid rows, input-overwrite protection and missing/empty files. External documentation blocks are preserved for learning and adaptation; their runtimes and dependencies have not all been executed. Arbitrary model output must be tested against the original requirements, even after Python syntax passes.

Recheck a recipient copy

From the extracted folder, run: "OG Companion.exe" --check-installation "%TEMP%\og-companion-check.json". The offline check writes its passed flag, title, teaching-entry count, documentation-edition count, source-browser controls, logo, foundation and resource location. Run source checks with: py -3 -m unittest -v og_companion_tests.py.

Relocation is checked on the development computer; a clean second computer remains a recipient acceptance check. Content and provider checks on 08/10/2026 used application source, provider documentation and live responses. Course submissions must follow the tutor’s rules on external assistance.